

Case 1: Sampling Based Motion Planning (OMPL)

MoveIt Task Constructor on the xArm 7, E-Waste Battery Extraction, Redback Operations

Prepared by Tarunjeet Singh, Project Lead

1. Purpose

This document records the first of three motion planning variants built on a shared MoveIt Task Constructor (MTC) pipeline for the xArm 7. Every stage in this case is solved by the OMPL planning pipeline, which searches the robot configuration space for a collision free path [1], [2], [3]. The result forms the baseline for the planner comparison.

2. Environment

Component	Version or detail
Host	Windows laptop running WSL2
Operating system	Ubuntu 24.04 LTS (Noble) in WSL2
Middleware	ROS 2 Jazzy Jalisco
Motion planning	MoveIt 2 (Jazzy binary packages)
Task planning	MoveIt Task Constructor (Jazzy binary packages)
Robot package	xarm ros2, jazzy branch, built from source in ~/xarm_ws
Simulation	xarm7_moveit_fake.launch.py with ros2_control fake hardware
Project package	xarm7_mtc_planners, node mtc_planner_compare (C++)

3. Scene and Task Structure

The task is a simulated vacuum pick and place of a phone battery. The same scene and stages are used in all three cases, so only the planner changes between them.

Item	Definition
Battery	Box 70 x 50 x 10 mm at (0.40, 0.00, 0.005) m in link_base, shown in orange
Drop zone	Pad 120 x 120 x 5 mm at (0.30, -0.30, 0.0025) m, shown in green
Planning group	xarm7 with IK frame link_eef, tool pointing down for all pose goals
Execution speed	Velocity and acceleration scaling factor 0.15
Hold between stages	2.0 s pause after every motion and every scene event

#	Stage	MTC stage type	Motion
0	Current state	CurrentState	Reads the live robot state and scene
1	Move above battery	MoveTo (pose)	To (0.40, 0.00, 0.20) m
2	Descend to battery	MoveRelative	0.15 m down in z
3	Vacuum ON	ModifyPlanningScene	Attach battery to link_eef

#	Stage	MTC stage type	Motion
4	Lift battery	MoveRelative	0.15 m up in z
5	Transfer to drop zone	MoveTo (pose)	To (0.30, -0.30, 0.20) m
6	Lower battery	MoveRelative	0.12 m down in z
7	Vacuum OFF	ModifyPlanningScene	Detach battery
8	Retreat	MoveRelative	0.12 m up in z
9	Return home	MoveTo (joints)	All seven joints to 0 rad

4. Planner Configuration

The PipelinePlanner solver sends each stage to the MoveIt planning pipeline named ompl, which uses the planner configuration shipped with the xArm MoveIt package [4], [5]. Velocity and acceleration scaling are applied to the solver so the resulting trajectories are slowed for recording.

```
if (type == "ompl") {
  // Case 1: sampling based planning through the OMPL pipeline
  s = std::make_shared<mtc::solvers::PipelinePlanner>(node);
}
s->setMaxVelocityScalingFactor(speed);
s->setMaxAccelerationScalingFactor(speed);
```

5. Parameter File Preparation

The MTC node needs the same robot model and planning parameters as move_group. In this setup the ros2 param dump command returned an empty file, and the parameters are stored as dotted names that must be nested for ROS to read them. The working procedure was:

- Fetch each move_group parameter individually through its get_parameters service and write them as nested YAML (build_params.py).
- Save the SRDF with ros2 param get /move_group robot_description_semantic and merge it into the same file.
- Receive the URDF from the /robot_description topic, which the node reads automatically.

The resulting file ~/xarm_ws/xarm7_params.yaml is reused by all three cases.

6. How to Run

Terminal 1: start the simulated xArm 7 with MoveIt and RViz.

```
source /opt/ros/jazzy/setup.bash
source ~/xarm_ws/install/setup.bash
ros2 launch xarm_moveit_config xarm7_moveit_fake.launch.py
```

Terminal 2: build the package and run this case.

```
cd ~/xarm_ws
MAKEFLAGS="-j1" colcon build --packages-select xarm7_mtc_planners
```

```
source install/setup.bash
ros2 run xarm7_mtc_planners mtc_planner_compare --ros-args \
  --params-file ~/xarm_ws/xarm7_params.yaml -p planner:=ompl
```

Optional parameters: speed (default 0.15), pause (default 2.0 s), countdown (default 5 s) and execute (default true). Every run appends one row to ~/mtc_results.csv.

7. Results

Runs 1 to 3 used the initial four stage task (above battery, descend, lift, home) at full speed. Runs 4 and 5 used the final ten stage task at a scaling factor of 0.15.

Run	Task	Plan (ms)	Cost	Exec (s)	Joint path (rad)
1	Initial, 4 stages	483.3	21.768	4.561	11.076
2	Initial, 4 stages	525.2	4.030	1.785	2.774
3	Initial, 4 stages	474.8	4.031	1.785	2.775
4	Final, 10 stages	430.6	11.715	14.380	5.927
5	Final, 10 stages	759.0	11.718	14.382	5.928

Metric definitions: planning time is the wall clock time of task.plan(), which includes reading the planning scene from move_group. Cost is the MTC solution cost from its default cost function, where lower is better. Execution time is the summed duration of all trajectories, excluding the demonstration pauses. Joint path length is the summed Euclidean distance between consecutive joint space waypoints.

8. Observations

- **Run to run variation:** on the identical initial task, run 1 produced a joint path about four times longer than runs 2 and 3 (11.08 rad against 2.77 rad) and an execution time about 2.6 times longer. This is the expected behaviour of a stochastic sampling planner.
- **Consistent on the final task:** runs 4 and 5 differed by less than 0.1 percent in cost, execution time and joint path, while planning time varied from 431 ms to 759 ms.
- **Path shape:** the descend, lift, lower and retreat stages are planned as free space motions, so the tool is not constrained to a straight vertical line.
- **Strength:** OMPL can route around obstacles, which makes it the most general option for transfer motions in a cluttered cell.

9. References

- [1] MoveIt Contributors, "Pick and place with MoveIt Task Constructor," MoveIt 2 Documentation. [Online]. Available: https://moveit.picknik.ai/main/doc/tutorials/pick_and_place_with_moveit_task_constructor/pick_and_place_with_moveit_task_constructor.html. Accessed: Sep. 28, 2026.
- [2] M. Görner, R. Haschke, H. Ritter, and J. Zhang, "MoveIt! Task Constructor for task level motion planning," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), Montreal, Canada, 2019, pp. 190–196.
- [3] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," IEEE Robotics & Automation Magazine, vol. 19, no. 4, pp. 72–82, Dec. 2012.
- [4] UFACTORY, "xarm_ros2: ROS 2 developer packages for robotic products from UFACTORY," GitHub, jazzy branch. [Online]. Available: https://github.com/xArm-Developer/xarm_ros2. Accessed: Sep. 28, 2026.
- [5] MoveIt Contributors, "MoveIt 2 Documentation." [Online]. Available: <https://moveit.picknik.ai/main/index.html>. Accessed: Sep. 28, 2026.
- [6] Open Robotics, "ROS 2 Documentation: Jazzy Jalisco." [Online]. Available: <https://docs.ros.org/en/jazzy/>. Accessed: Sep. 28, 2026.